

Western Norway
University of
Applied Sciences

Random Rounding of Semidefinite Programs

Chapter 6

Sven-Olai Høyland

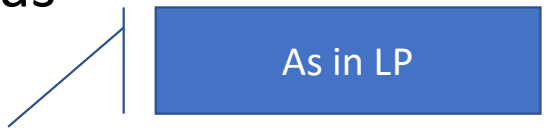
Mathematics

- The first slides contain mathematics that many of you may not be familiar with
- Do not worry
- For the exam, the important things is about coloring algorithms, starting on slide 18.

Semidefinite programming

- Definition 6.1. A matrix $X \in \mathcal{R}^{n \times n}$ is positive semidefinite iff for all $y \in \mathcal{R}^n, y^T X y \geq 0$

- A semidefinite program (SDP) has
  - Linear objective function
  - Linear constraints
  - A square symmetric matrix of variables can be constrained to be positive semidefinite



Example

- A square symmetric matrix that is positive semidefinite

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

For all vectors $[a, b] \in R^n$

$$[a, b] \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \end{bmatrix} = [a, b] \cdot \begin{bmatrix} a \\ b \end{bmatrix} = a^2 + b^2 \geq 0$$

Example

- Maximize or minimize $\sum_{i,j} c_{ij} x_{ij}$
- Subject to
  - $\sum_{i,j} a_{ij} x_{ij} = b_k, \qquad \forall k$
  - $x_{ij} = x_{ji}, \qquad \forall i, j$
  - $X = \left(x_{ij}\right) \succcurlyeq 0$ (positive semidefinite)

- Will assume that SDP can be solved exactly in polynomial time (since that applies for the algorithms we are looking at)

Vector programming

- Maximize or minimize $\sum_{i,j} c_{ij}(v_i \cdot v_j)$
- Subject to
  - $\sum_{i,j} a_{ij}(v_i \cdot v_j) = b_k, \qquad \forall k$
  - $v_i \in \mathcal{R}^n, \qquad i = 1, \ldots, n$

- Note: The variables are vectors ($v_i \in \mathcal{R}^n$)
- SDP and Vector program are equivalent

Finding large cuts

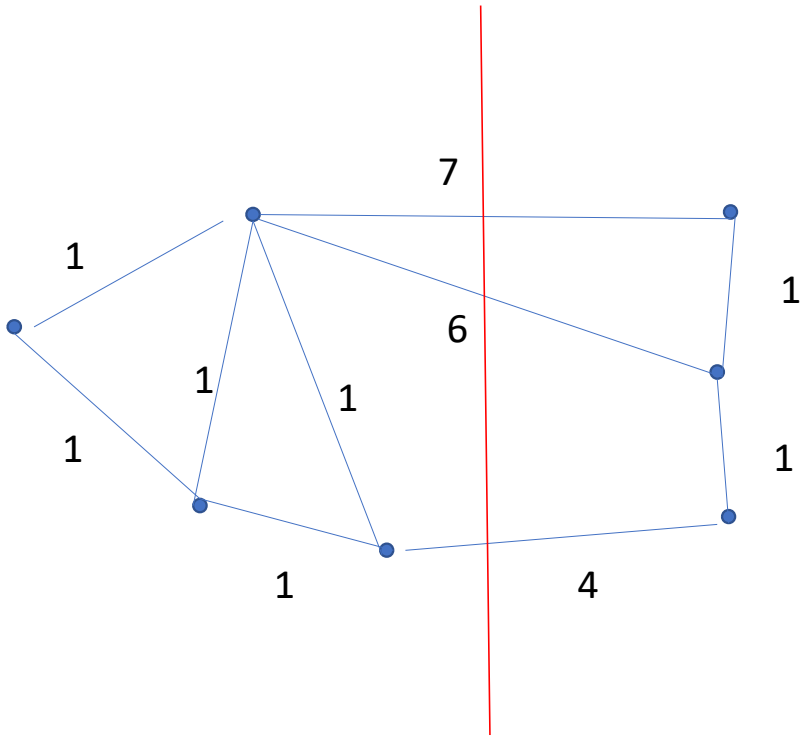
- Will find improved algorithm for MAX CUT using SDP
- First a quick review of what we did in Chapter 5

MAX CUT

- Undirected graph $G = (V, E)$
- Nonnegative weight $w_{ij} > 0$ for each edge $(i, j) \in E$

- Goal: Partition the vertices in to parts U and $W = V - U$ so as to maximize the weight of the edges where the two endpoints are in different parts.

Example



Algorithm

- Place each vertex $v \in V$ into U independently with probability ½

Theorem 5.3

- If we place each vertex $v \in V$ into U independently with probability ½, then we obtain a ½ -approximation algorithm for the maximum cut problem

Integer program for MAX CUT

- Relax integer program to vector program
- Maximize $\frac{1}{2}\sum_{(i,j)\in E} w_{ij}(1 - y_i \cdot y_j)$
- Subject to $y_i \in \{-1, +1\}, \qquad i = 1, \ldots, n$

- Consider $U = \{i: y_i = -1\}$ and W$= \{i: y_i = +1\}$
- Note: If edge (i, j) is in the cut $y_i y_j = -1$, and if the edge is not in the cut we have $y_i y_j = 1$

Relaxatioin

- Maximize $\frac{1}{2}\sum_{(i,j)\in E} w_{ij}(1 - v_i v_j)$
- Subject to
  - $v_i \cdot v_i = 1, \qquad i = 1, \dots n$
  - $v_i \in \mathcal{R}^n, \qquad i = 1, \dots n$

- Comments:
  - Relaxation since we can take feasible solution y and produce feasible solution to this program by setting $v_i = (y_i, \dots, 0)$. Then $v_i \cdot v_j = y_i y_j$
  - Hence $Z_{VP} \geq OPT$

Algorithm

- Solve the vector program in poly time
- Pick a random vector $r = (r_1, \dots, r_n)$ by drawing each component from $\mathcal{N}(0,1)$
- Iterate through the vertices and put $i \in U$ if $v_i \cdot r \geq 0$ and $i \in W$ otherwise

How good is the solution

- Theorem 6.9. The algorithm on previous slide is a 0.878-approximation algorithm for the maximum cut problem

Possible to do better?

- Theorem 6.10. If there is an α-approximation algorithm for maximum cut problem with $\alpha > \frac{16}{17} \approx 0.941$, then P = NP

- Comment. It is the best we can achieve if unique game conjecture holds

- Chapter 6.3 and 6.4 not in curriculum

Coloring 3-Colorable Graphs

- Previous chapter
  - We can with high probability 3-color a δ-dense 3-colorable graph

Definition 6.19

- A function $g(n) = \tilde{O}(f(n))$ if there exists some constant $c \geq 0$ and some n_0 such that for all $n \geq n_0, g(n) = O(f(n) \log^c n))$.

2-colorable graphs

- Can be colored it polynomial time
- Algorithm
  - Assume graph is connected
  - Chose an arbitrary vertex v
  - Color v blue
  - S={v}
  - while there are uncolored vertices do
    - Select a vertex with at least one neighbor in S (always possible since graph is connected)
    - Color it with color different from its neighbor(s) in S

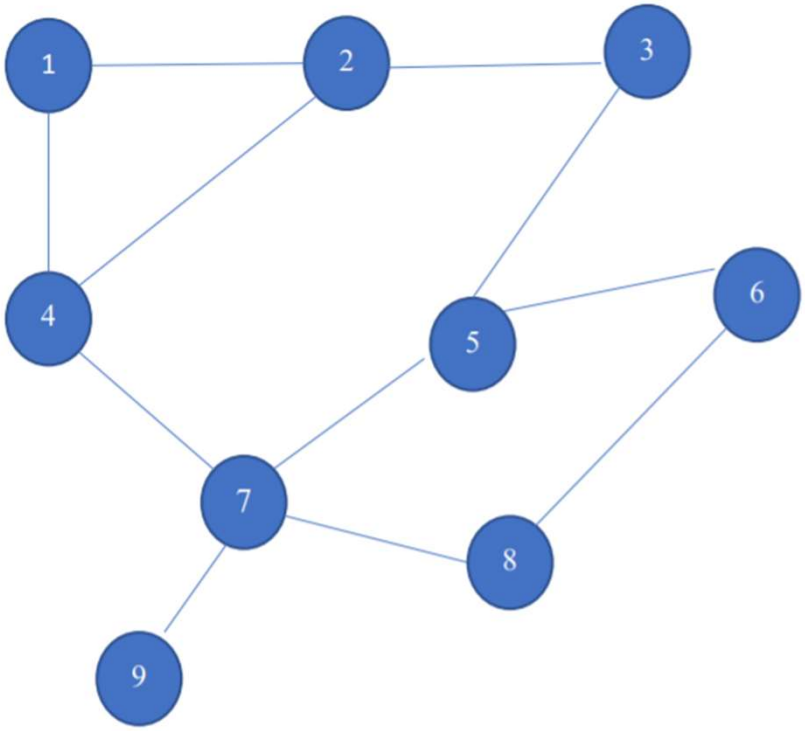
Arbitrary graphs

- Let Δ be the maximum degree in the graph
- Then its simple to color it with $(\Delta + 1)$ colors
  - Can color the vertices in any order. When coloring a vertex, look at it neighbors and since there are at most Δ of them, there at least one available color

Algorithm for coloring a 3-colorable graph with $O(\sqrt{n})$ colors

- While there exists a vertex in the graph with degree at least $\sqrt{n}$
  - Chose a vertex in the graph with degree at least $\sqrt{n}$
  - Pick 3 new colors.
  - Color the vertex with one
  - Use the 2-coloring algorithm to color its neighbors with the other two colors (possible since the graph is 3-colorable)
  - Remove these vertices from the graph
- Use algorithm that colors a graph with $(\Delta + 1)$ colors to color the remaining vertices

Example

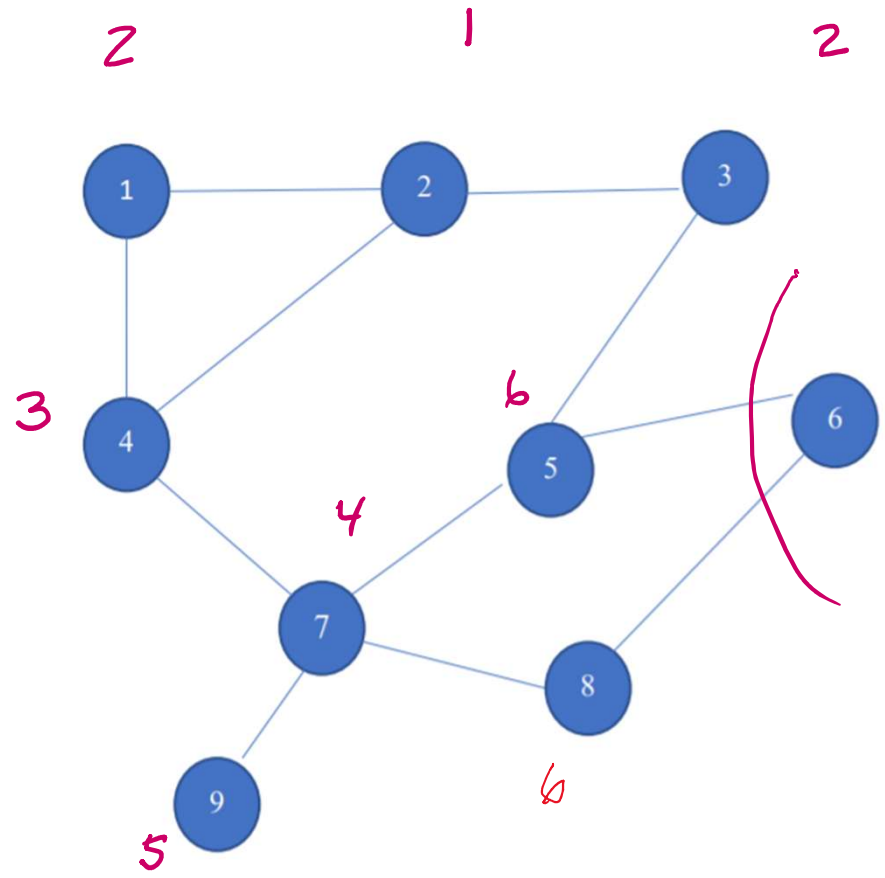


Example

The numbers next to nodes are colors

1. pick 2
2. pick 7

first loop



Last step of alg but only one vertex

Theorem 6.20

- The algorithm above colors any 3-colorable graph with at most $4\sqrt{n}$ colors

- Proof: First step can happen at most $\frac{n}{\sqrt{n}}$ times since we remove at least $\sqrt{n}$ vertices each time. Use 3 colors each time for a total of $3\sqrt{n}$. The final step uses at most $\sqrt{n}$ colors. In total $4\sqrt{n}$ colors

Vector program

- Minimize λ
- Subject to
  - $v_i \cdot v_j \leq \lambda, \qquad \forall (i,j) \in E$
  - $v_i \cdot v_i = 1, \qquad \forall i \in V$
  - $v_i \in \mathcal{R}^n, \qquad \forall i \in V$

- Lemma 6.21: For any 3-colorable graph, there is a feasible solution with $\lambda = -\frac{1}{2}$. Our book shows a solution. Hence we know $\lambda \leq -\frac{1}{2}$

Semicoloring

- A semicoloring is a coloring of nodes such that at most $\frac{n}{4}$ edges have endpoints with the same color.



- At least $\frac{n}{2}$ vertices are colored such that the endpoints are colored differently (Each edge as two endpoints)

How do we find a semicoloring?

- Use a randomized algorithm
- Solve the vector program
- Choose $t = 2 + \log_3 \Delta$ random vectors
- The t vectors define 2^t different regions into which the vectors v_i can fall.
  - Use different color for each region

Theorem 6.23

- This coloring produces a semicoloring of $4\Delta^{\log_3 2}$ colors with probability at least ½.

Algorithm, First try

- while there are vertices left
  - Semicolor the remaining graph with k colors
  - Remove the vertices that are correctly colored

- Comment: Since half of the vertices are correctly colored in each iteration, we have at most $\log_2 n$ iterations. Hence, we use at most $\mathrm{k} \log_2 n$ colors

Problem

- If we use n as an upper bound of Δ, the algorithm uses $\tilde{O}\left(n^{\log_3 2}\right)$ colors
- Since $\log_3 2 \approx 0.631$, this is worse than our previous algorithm which uses $4\sqrt{n} = 4n^{0.5}$ colors

- Solution. Use ideas from first algorithm in combination with semicoloring

Algorithm

- $\sigma = n^{\log_6 3} \approx 0.61$
- While there exists a vertex in the graph with degree at least σ
  - Chose a vertex in the graph with degree at least σ
  - Pick 3 new colors.
  - Color the vertex with one
  - Use the 2-coloring algorithm to color its neighbors with the other two colors
  - Remove these vertices from the graph
- Use algorithm for semicoloring a graph to color the remaining vertices with $O(\sigma^{\log_3 2})$ new colors

Theorem 6.24

- The algorithm above semicolors a 3-colorable graph with $O(n^{0.387})$ colors with probability at least ½.